

Strutture dati

Array

Come raccogliere piu valori in una sola variabile.

Cosa e un array

Un array e una variabile che contiene piu valori. Pensalo come una fila di caselle numerate.

```
<?php
$frutti = ["mela", "pera", "banana"];
```

Ogni elemento ha una posizione, chiamata **indice**. In PHP il primo indice e `0`.

Leggere un elemento

```
<?php
$frutti = ["mela", "pera", "banana"];

echo $frutti[0];
```

Output:

```
mela
```

`$frutti[1]` contiene `"pera"`.

Aggiungere elementi

Puoi aggiungere un valore in fondo all'array con `[]` .

```
<?php
$frutti = ["mela", "pera"];
$frutti[] = "banana";

print_r($frutti);
```

`print_r` mostra una struttura in modo leggibile, utile mentre studi.

Contare elementi

```
<?php
$frutti = ["mela", "pera", "banana"];

echo count($frutti);
```

Output:

```
3
```

Scorrere un array

Con `foreach` puoi lavorare su ogni elemento.

```
<?php
$frutti = ["mela", "pera", "banana"];

foreach ($frutti as $frutto) {
    echo $frutto . "\n";
}
```

A ogni giro, `$frutto` contiene un valore diverso.

Modificare un elemento

Puoi cambiare un valore usando il suo indice.

```
<?php
$frutti = ["mela", "pera", "banana"];
$frutti[1] = "arancia";

print_r($frutti);
```

Ora al posto di `pera` trovi `arancia`.

Errore comune: dimenticare che si parte da zero

Se un array ha tre elementi, gli indici sono `0`, `1` e `2`.

```
<?php
$frutti = ["mela", "pera", "banana"];

echo $frutti[3];
```

`$frutti[3]` non esiste. Il terzo elemento è `$frutti[2]`.

A cosa servono davvero gli array

Gli array sono utili quando hai più valori dello stesso tipo: nomi, prezzi, voti, messaggi, prodotti.

```
<?php
$prezzi = [10, 20, 15];
$totale = 0;

foreach ($prezzi as $prezzo) {
    $totale += $prezzo;
}

echo $totale;
```

Qui l'array permette di calcolare il totale senza creare tre variabili separate.

Prova tu

Crea un array con tre nomi. Poi usa `foreach` per stampare `Ciao` seguito da ogni nome.

Array associativi

Come usare chiavi testuali per rappresentare dati strutturati.

Dalle posizioni ai nomi

Negli array indicizzati leggi i valori con numeri come `0` e `1`. Negli array associativi usi chiavi testuali.

```
<?php
$utente = [
    "nome" => "Luca",
    "eta" => 28,
    "email" => "luca@example.com",
];
```

Qui `"nome"`, `"eta"` ed `"email"` sono chiavi.

Leggere un valore

```
<?php
$utente = [
    "nome" => "Luca",
    "eta" => 28,
];

echo $utente["nome"];
```

Output:

Luca

Modificare un valore

```
<?php
$prodotto = [
    "nome" => "Quaderno",
    "prezzo" => 3.50,
];

$prodotto["prezzo"] = 3.20;
```

Stai cambiando solo il valore collegato alla chiave `"prezzo"`.

Aggiungere una chiave

```
<?php
$prodotto = [
    "nome" => "Quaderno",
];

$prodotto["disponibile"] = true;
```

Scorrere chiavi e valori

```
<?php
$utente = [
    "nome" => "Sara",
    "eta" => 31,
];

foreach ($utente as $chiave => $valore) {
    echo "$chiave: $valore\n";
}
```

Gli array associativi sono utili per rappresentare profili, prodotti, impostazioni e dati che hanno un nome preciso.

Un prodotto come array associativo

Quando i dati descrivono una stessa cosa, le chiavi rendono il codice più leggibile.

```
<?php
$prodotto = [
    "nome" => "Penna",
    "prezzo" => 1.20,
    "disponibile" => true,
];

if ($prodotto["disponibile"]) {
    echo $prodotto["nome"] . " costa " . $prodotto["prezzo"] . " euro";
}
```

Leggendo le chiavi capisci subito che cosa rappresenta ogni valore.

Controllare se una chiave esiste

Prima di leggere una chiave che potrebbe mancare, puoi usare `isset`.

```
<?php
if (isset($utente["email"])) {
    echo $utente["email"];
}
```

Oppure puoi usare `??` per dare un valore predefinito.

```
<?php
$email = $utente["email"] ?? "email non disponibile";
```

Errore comune: usare il nome della chiave senza virgolette

```
<?php
echo $utente[nome];
```

Scrivi invece:

```
<?php
echo $utente["nome"];
```

Le virgolette fanno capire a PHP che `nome` è una chiave testuale.

Date e tempo

Come rappresentare date, orari e intervalli nei programmi PHP.

Perche le date richiedono attenzione

Una data sembra semplice, ma può includere giorno, mese, anno, ora, minuti, secondi e fuso orario. PHP offre `DateTime` per gestirle in modo più ordinato.

Creare una data

```
<?php
$oggi = new DateTime();

echo $oggi->format('d/m/Y');
```

`new DateTime()` crea un oggetto con data e ora attuali. `format` decide come mostrarlo.

Creare una data specifica

```
<?php
$scadenza = new DateTime('2026-06-15');

echo $scadenza->format('d/m/Y');
```

Output:

```
15/06/2026
```

Fusi orari

Il fuso orario dice a quale zona del mondo si riferisce l'orario.

```
<?php
$roma = new DateTime('now', new DateTimeZone('Europe/Rome'));

echo $roma->format('H:i');
```

Usare il fuso giusto evita risultati strani, soprattutto nei siti usati da persone in paesi diversi.

Timestamp

Un timestamp è un numero che rappresenta un momento nel tempo.

```
<?php
echo time();
```

È utile per confronti tecnici, ma per codice leggibile spesso `DateTime` è più chiaro.

Confrontare date

```
<?php
$soggi = new DateTime('2026-05-02');
$scadenza = new DateTime('2026-05-10');

if ($soggi < $scadenza) {
    echo "Sei ancora in tempo";
}
```

PHP confronta i due momenti e decide quale viene prima.

Aggiungere o togliere tempo

Con `modify` puoi spostare una data avanti o indietro.

```
<?php
$scadenza = new DateTime('2026-05-02');
$scadenza->modify('+7 days');

echo $scadenza->format('d/m/Y');
```

Output:

```
09/05/2026
```

Questo è utile per scadenze, promemoria e periodi di prova.

Differenza tra due date

```
<?php
$inizio = new DateTime('2026-05-02');
$fine = new DateTime('2026-05-10');

$differenza = $inizio->diff($fine);

echo $differenza->days;
```

Output:

8

`diff` restituisce un intervallo. La proprietà `days` contiene il numero totale di giorni tra le due date.

Errore comune: formato ambiguo

Date come `05/06/2026` possono essere lette come 5 giugno o 6 maggio, a seconda del contesto. Quando crei date nel codice, preferisci formati chiari come `2026-06-05`.

Stringhe

Come creare, combinare e manipolare testo in PHP.

Cosa è una stringa

Una stringa è testo: un nome, un messaggio, una email, una frase.

```
<?php
$nome = "Luca";
$saluto = 'Ciao';
```

Puoi usare virgolette doppie o singole.

Virgolette doppie e singole

Con le virgolette doppie, PHP legge le variabili dentro la stringa.

```
<?php
$nome = "Luca";
echo "Ciao $nome";
```

Output:

Ciao Luca

Con le virgolette singole, il testo resta letterale.

```
<?php
$nome = "Luca";
echo 'Ciao $nome';
```

Output:

Ciao \$nome

Concatenazione

Per unire stringhe usa il punto `.`.

```
<?php
$nome = "Sara";
$messaggio = "Ciao " . $nome . "!";

echo $messaggio;
```

Funzioni comuni

PHP offre molte funzioni per lavorare con il testo.

```
<?php
$testo = "Manuale PHP";

echo strlen($testo);
```

`strlen` conta i caratteri in byte. Per testi con lettere accentate puo servire `mb_strlen`, se l'estensione `mbstring` e disponibile.

Altre funzioni utili:

- `strtolower($testo)` porta in minuscolo
- `strtoupper($testo)` porta in maiuscolo
- `trim($testo)` toglie spazi all'inizio e alla fine
- `str_replace("PHP", "web", $testo)` sostituisce testo

Testo ricevuto dagli utenti

Quando mostri testo inserito da un utente in una pagina HTML, usa `htmlspecialchars`.

```
<?php
echo htmlspecialchars($nome, ENT_QUOTES, 'UTF-8');
```

Questo evita che testo pericoloso venga interpretato come HTML.

Leggere una parte di testo

Puoi prendere solo una parte di una stringa con `substr`.

```
<?php
$codice = "ORD-2026-15";
$prefisso = substr($codice, 0, 3);

echo $prefisso;
```

Output:

```
ORD
```

Il secondo numero indica da dove partire. Il terzo indica quanti caratteri prendere.

Controllare se un testo contiene una parola

```
<?php
$email = "luca@example.com";

if (str_contains($email, "@")) {
    echo "Sembra una email";
}
```

Questo non basta per validare davvero una email, ma mostra come controllare la presenza di una parte di testo.

Errore comune: dimenticare gli spazi nella concatenazione

```
<?php  
$nome = "Luca";  
echo "Ciao" . $nome;
```

Output:

```
CiaoLuca
```

Lo spazio va scritto esplicitamente:

```
<?php  
echo "Ciao " . $nome;
```